CONSIGNMENT BOUTIQUE

Classic Consigns by KYS

Point of Sale & Store Management · Classic Consigns by KYS

User & Admin Guides

USER GUIDE

1. Signing in
2. Point of Sale (POS)
   - Ringing up a sale
   - Refunds
   - Notes
3. Inventory
   - Views & sorting
   - The scan box
   - Adding an item
   - Quick Intake (camera-first, for the tablet)
   - Editing an item
   - Bulk actions
   - Labels & KYS tags
   - Aging alerts
4. Consignors
   - Appointments (new-consignor booking)
   - Layaway
   - Borrowed (garment agreements)
   - Sip & Shop
   - Consignor detail page
5. Customers
   - Kiosk sign-in (digital guest book)

ADMIN & OPERATIONS GUIDE

1. Architecture at a glance
2. Hosting on Railway
   - Deploying changes
3. Environment variables
4. First-run setup (new deployment)
5. Accounts & roles
   - Resetting a password
6. Store configuration (Settings)
   - Admin — employee work log (/admin)
   - Consignment appointments (/book public page)
   - Layaway & Borrowed agreements
   - Sip & Shop events
   - Kiosk customer sign-in (/welcome)
7. Integrations
   - Clover (card payments)
   - Resend (email)
   - Anthropic / Claude (AI item entry)
   - Shopify (webhooks, inbound)
   - Outbound webhooks / API keys
8. Importing data
9. Data export & reporting
10. Backups & data safety

6. Payouts
7. Contracts
7a. Admin — employee hours & payouts
8. Store Insights & Reports
9. Online shop
10. Consignor & Dealer portals
11. Importing existing data
Quick tips
11. Direct database access
12. Local development
13. Troubleshooting
14. Security checklist

Updated **November 2026** · Staff User Guide and Owner/Admin Guide

📍 7214 Harford Rd, Baltimore, MD 21234 · 📞 443.841.9730 · ✉ kimberly@classicconsigns.com

CLASSIC CONSIGNS BY KYS

User Guide

For store staff

Classic Consigns by KYS — User Guide

A guide for store staff (owners, managers, and employees) using the Classic Consigns by KYS consignment platform day to day.

> **Your store address:** your platform lives at your own web address (e.g. `https://your-store.up.railway.app`). Replace "your site" below with it.

---

1. Signing in

1. Go to **your site** → you'll land on the **Sign in** page.
2. Enter your **email** and **password** → **Sign In**.
3. You'll arrive at the **Point of Sale** screen.

Forgot your password? Ask your store owner/admin — they can reset it (see the Admin Guide → "Resetting a password").

Dark / light mode: at the bottom of the left sidebar, use **Dark mode / Light mode** to switch themes. Your choice is remembered on that device.

---

2. Point of Sale (POS)

The POS is where you ring up sales.

Ringing up a sale

1. **Find items** — type a name or SKU in the search box, or **scan a barcode**. - **Scanning:** just scan the item's label — it's **added straight to the cart** and you'll see a green "Added: …" confirmation. Scan the next item, and so on. - **Typing a name:** results appear below; click one to add it to the cart.
2. **Review the cart** on the right — remove a line with the trash icon.

3. **Discount (optional):** type a dollar amount in the Discount box.
4. **Loyalty points (optional):** - In the **Loyalty Points** box, type the customer's **email** → **Find**. - Their available points show. Enter how many points to **redeem** (100 points = $1). The discount is applied automatically.
5. **Choose payment: Card** (Clover) or **Cash**.
6. Click **Charge $X.XX**.
7. A **receipt** pops up — **Print Receipt** or Close. Points are awarded to the customer automatically (1 point per $1 spent) when a customer is attached.

Refunds

At the top of the POS screen, switch from **Sell** to **Refund** to undo a sale. 1. **Scan the sold item** (or type its SKU) in the box. A refund card appears on the right showing what it sold for and, for a consigned item, the **consignor credit that will be reversed**. 2. Choose how the item comes back: - **Refund & relist** — the item goes back on the floor as **Active**, ready to sell again. - **Refund & return to consignor** — the item is marked **Returned**. 3. Either way, the consignor's **Ledger** gets a matching **debit** (a red `ADJUSTMENT`) that **reverses the sale credit**, and their balance drops by the same amount.

Notes: - Only a **Sold** item can be refunded; scanning anything else shows why it can't. - The reversal uses the **price the item actually sold for** (not its current price). If the consignor was already paid out, their balance can go negative — that simply means they owe that amount back. - The **customer's** money (card/cash) is returned on your payment terminal; the refund here handles the inventory and the consignor's ledger.

Notes

- Only **Active** items can be sold. Sold/returned items won't scan into a sale.
- Card payments create an order in your Clover account; the actual card charge happens on your Clover device/terminal.

---

3. Inventory

Manage every item in the store. Open **Inventory** from the sidebar.

Views & sorting

- Top-right, switch between **List**, **Cards**, and **Compact** views.
- In **List** view, click any column header (Item, Brand, Consignor, Price, Added) to sort; click again to reverse.
- **Status tabs** (Active / Sold / Returned / Expired) filter what you see.
- **Search** by title, brand, SKU, or consignor name.

The scan box

At the top there's a **Scan** box. Scan a barcode or SKU (or type it and press Enter) and that item **opens for editing** immediately — handy for quick price checks or edits at the counter.

Adding an item

1. **Inventory → Add Item**.
2. Fill in the details: title, brand, size, condition, price, consignor, category.
3. **AI Item Entry (optional):** drop a photo and the assistant fills in the title, brand, category, condition, size, color, description, and a suggested price for you to review.
4. **Photos:** drag-and-drop photos; the first is the main image.
5. **Custom SKU (optional):** type your own SKU (e.g. `CC-DRESS-001`) or leave it blank to auto-generate.
6. **Barcode:** scan a barcode into this field (or leave blank to use the SKU).
7. **List on public online shop:** toggle on to show the item at your `/shop`.
8. **Save** → you'll get the SKU and a **Print Label** button.

Quick Intake (camera-first, for the tablet)

For adding many items fast — especially on the **store tablet** — use **Inventory → Quick Intake** (or **Quick intake** on a consignor's Items tab to start with them pre-selected). 1. **Pick the consignor once** and tap **Start** — they stay locked for the whole batch, so you never re-select. 2. Tap **Photograph the item** → the tablet **camera** opens → snap the item. 3. The **AI reads the photo** and fills in the title, price, condition, brand, size, and description. **Add angle** to attach more photos of the same item. 4. **Glance and adjust** (mainly the price), then **Save & next item** — it saves to that consignor and instantly resets for the next piece. 5. Everything you add shows in **Added this session** with a **Print KYS tag** button for each, plus a link to the consignor's items.

So a whole bag becomes **snap → confirm → save**, repeated. (The AI needs `ANTHROPIC_API_KEY` set — see the Admin Guide.)

Editing an item

Click any item (row or card) in Inventory to open the quick **Edit** window (title, SKU, barcode, brand, size, condition, price, cost, location, status), with **Print**, **KYS Tag**, and **Delete**. For the complete form — the same one as Add Item, with **photos, AI photo auto-fill, color, description, category, online listing**, etc. — click **Full edit** in that window, or edit from a consignor's **Items** tab (the pencil opens the full editor directly). After saving, you're returned to wherever you came from (e.g. back to the consignor you were reviewing). Deletion is blocked for sold items.

Bulk actions

Select items with the checkboxes (or "select all"), then use the blue bar: - **Print labels** — prints a scannable label for every selected item. - **Markdown %** — reduce the price of all selected items by a percentage. - **List online / Unlist** — show/hide on the public shop in bulk. - **Mark Returned** — change status in bulk. - **Delete** — remove selected items (sold items can't be deleted).

Labels & KYS tags

Two print formats, both with a **scannable Code128 barcode** (the item's barcode, or its SKU): - **KYS Tag** — a branded **2" × 2"** hang tag with the **Classic Consigns by KYS logo**, a **short description** (the item's title), the **Size**, **Price**, **Consignor**, the SKU barcode, and SKU text. Designed to print on your **own printer** (plain cardstock or tag sheets). Print it from the **Add-Item** screen, an item's **Edit** window ("KYS Tag"), the inventory bulk bar (**Print KYS tags**), or a consignor's page (**Print all tags** prints a sheet of tags for all their active items). Bulk printing lays the tags out in a grid with faint cut guides. - **Blank price option:** tick **"Blank price"** (on a consignor's page) or **"Blank $"** (in the inventory bulk bar) before printing to leave the price as a **write-in line** (`$_____`) instead of printing the value — handy for hand-writing or marking down prices. Size and Consignor still print. - **Label** — the compact **2" × 1"** thermal label (title, brand/size, price, barcode). Print from the Edit window ("Label") — handy for a thermal label printer.

Aging alerts

On the Active tab, items expiring within 14 days are flagged at the top so you can mark them down or return them.

---

4. Consignors

The people who bring you items to sell. Open **Consignors** from the sidebar.

- **Views / sorting / search / multi-select** work the same as Inventory.
- **Add** a consignor with the **Add** button (name, contact, split %).
- **Click a consignor** to edit their name, contact, **split %**, and **portal access**, or open **Details** for their full items / earnings / payout history.
- **Bulk actions:** set split %, enable/disable portal access, delete.
- **Balance** is what you currently owe each consignor. It goes up whenever one of their items **sells** — either through the POS or by **marking an item SOLD** from the Edit window. Once a consignor's balance is above $0 they appear on the **Payouts** page. (Changing a sold item back to Active reverses the credit.)
- When adding or editing a consignor you can set their **preferred payout method** (Check/Cash/Zelle/Cash App/ACH) with the matching details, and an **"If items don't sell"** preference: **Pick-up**, **Donate**, or **Continue consigning (30 days)**.

Appointments (new-consignor booking)

Next to **Customers** and **Consignors** there's an **Appointments** tab for booking **new/prospective consignors** into drop-off slots.

- **Schedule:** Saturdays only, **12:00–3:00 PM**, in four 40-minute slots (12:00, 12:40, 1:20, 2:00) — up to **4 appointments per day**.

- **Public booking page:** share **your site** `/book` (use **Copy booking link**). Prospective consignors pick an open Saturday slot and enter their name, email, and phone — no login needed. Taken slots show as unavailable. The page lists your rules (10-item first-visit limit; shoes & jewelry not counted).
- **Staff view:** the Appointments tab shows each upcoming Saturday with its four slots and who's booked. You can **Add** a booking manually, **cancel** one (trash icon), or **Block day** / **Unblock day** to close a Saturday (e.g. a holiday) so no new appointments can be booked.
- **Any day / any time (weekday exceptions & 5th/6th appointments):** click **New appointment** (top-right) to schedule someone on **any date and any time** — e.g. a consignor who can't do Saturday and wants Friday at 4:30 PM, or an extra person beyond the four Saturday slots. Pick the date and time, enter their details, and they get the **same email confirmation**. These show up under **"Also booked (extra times)"** on that Saturday (if it's a Saturday) or in the **"Other appointments"** section for weekdays. There's also an **"Add extra appointment (any time)"** link on each Saturday for quick extra bookings.
- **Email confirmations:** when someone books with an email address, they automatically get a confirmation email with the date, time, location, and the item/jewelry reminders. (If they don't enter an email, no confirmation is sent — but you'll still be notified, see below.)
- **You get notified of every booking:** each time someone books (online or added by staff online), the store receives a **"New appointment" email** with the person's name, time, phone, and email. It goes to your store email (set in **Settings → Store Information**), or to `kimberly@classicconsigns.com` if no store email is set. The Appointments page also **auto-refreshes**, so new bookings appear on their own within about 30 seconds.
- **Turn an appointment into a consignor:** once someone shows up and you're ready to set them up, click **Add as consignor** on their appointment. It creates a consignor record from their name/email/phone (50% split), removes the appointment, and opens the new consignor's page. If a consignor with that email already exists, it opens that one instead of making a duplicate.

Layaway

The **Layaway** tab (next to Appointments) tracks layaway agreements.

- **New Layaway:** enter the buyer's info, the item(s), total price, the $5 layaway fee, and the four payments (deposit today, then day 15, day 30, and the final at day 45). If you enter the buyer's **email**, they're automatically sent their **layaway agreement with the full payment schedule** when you save.
- **Automatic reminders:** the buyer gets an email reminder at **15, 30, and 45 days** for the next payment due (and the balance remaining). Reminders stop once the layaway is fully paid.
- **Recording payments:** open a layaway (pencil) and use **Record a payment** to add what the buyer paid; the balance updates and it marks **Completed** when paid in full. You can also set the status to Cancelled.
- The top cards show how many are active and the total outstanding balance.

Borrowed (garment agreements)

The **Borrowed** tab tracks items lent out (e.g. for events).

- **New Agreement:** enter the borrower's name, ID, phone, email, the dates borrowed/return, the item list, condition notes, and who's responsible for dry cleaning. If you enter the borrower's **email**, they're automatically

sent the **agreement listing the items** and the terms.

- **Scan to add items (easiest):** in the item list area there's a **scan box** — scan each borrowed piece's **KYS tag** (or type its SKU and press Enter) and the app pulls that item's **brand, size, price, and consignor** straight from inventory and adds a line automatically. No more typing it all out.
- **Add from a photo (AI):** for an item that isn't in inventory, click **Add from photo (AI)**, snap/upload a picture, and it fills in a line (title, brand, size, suggested price) for you to tweak.
- You can still type or edit any line by hand in the box.
- **Return reminder:** if a return date and email are set, the borrower gets a reminder email when the item is due back.
- Mark an agreement **Returned** (pencil → Status) when the items come back.

Sip & Shop

The **Sip & Shop** tab is a **calendar** for scheduling private Sip & Shop events (they run **7:00–10:00 PM**).

- **Schedule an event:** tap any day on the calendar (or **New Event**) and enter the **host's name, phone, email**, and **how many guests** are joining. Events show on the calendar as a pink name chip; tap a name to edit.
- **Host confirmation email:** if you add the host's email, they're automatically emailed a confirmation with the **date, the 7–10 PM time, their guest count**, and the perks (two wines, charcuterie, up to 15 guests, 15% store credit).
- **Reminder:** the host gets a reminder email a couple of days before the event.
- **Customers can book themselves:** share **your site** `/sip-and-shop` (use **Copy booking link**). Customers pick their own date, enter their info and guest count, and submit — it drops straight onto your calendar, emails them a confirmation, and emails you a "New Sip & Shop request" alert.
- Edit an event to change the date/guests or set it **Completed** / **Cancelled**.

Consignor detail page

Shows KPI cards (balance, earned, active, sold), and tabs for **Items**, **Ledger** (every credit/debit), and **Payouts**. You can **issue a payout** here (Check / Cash / ACH).

- **Download report:** the **Download report** button opens a clean, printable summary of that consignor — their contact/split, the KPI totals, every item (with SKU, price, status), and their payout history. Use your browser's **Print** dialog to print it or **Save as PDF**.
- **Email inventory (onboarding):** the **Email inventory** button emails the consignor a welcome message with **their list of items** (title, SKU, size, price) and a link to their **Consignor Portal** so they can track items going forward. You'll be asked which email to send to:
- Enter **your own email** to **test it first** — a test send makes **no changes** to the consignor's portal.
- Send to the **consignor's own email** to go live — this also **enables their portal** and includes a temporary login password in the email so the link works right away.
- Editing an item from here (or adding one) returns you right back to this consignor's page when you're done.

- **Emails sent (history):** an **Emails sent** panel lists every inventory and contract email sent for this consignor, with the **date & time**, type, who it went to, and whether it was a real send, a **test**, or **failed** — so you always have a record of what was sent and when.
- **Notes:** there's a **Notes** box on each consignor's page — type anything you want to remember (preferences, reminders, special arrangements) and **Save notes**. It's kept with the consignor.
- **Item notes:** open an item's **Edit** window (pencil on the Items tab) and use the **Note** field to jot anything specific to that item. Saved with the item.
- **Consignor payout preview:** the Edit window shows the **consignor's payout** (price × their split %) live as you change the price. If you edit the price of an item that's already **SOLD**, saving automatically re-adjusts the consignor's credit and balance to match. (The **Sync sold items** button on the Payouts page also corrects any sold items whose price changed earlier.)
- **Returns on the ledger:** when you mark an item **Returned** — the pencil → **Status** → **Returned** on the Items tab, or **Mark Returned** as a bulk action in Inventory — the consignor's **Ledger** records it as *Returned: "<item>" to consignor* at **$0.00** (a return moves no money, so the balance is unchanged). It's logged once, the first time the item becomes Returned.

---

5. Customers

Your loyalty members. Open **Customers** from the sidebar.

- **Views / sorting / search / multi-select** as above.
- **Add** a customer (name, email, phone).
- **Click a customer** to edit name, email, phone, and **loyalty points**, or delete them.
- **Purchase history:** the customer's detail panel shows their **visits**, **lifetime spend**, and a full **Purchase history** — every item they've bought, grouped by sale (date and amount), with each item's price and which consignor it came from.
- **Bulk actions:** add points to many at once, or delete.
- Customers earn **1 point per $1** spent (when attached to a sale) and redeem **100 points = $1** at the POS.

Kiosk sign-in (digital guest book)

Replace the paper guest book with a tablet. Share **your site** `/welcome` (use the **Kiosk link** button on the Customers page to copy it), open it on the tablet in full-screen, and set the tablet to **kiosk / Guided Access mode** so customers stay on that page.

- Customers enter their **name, email, phone**, and optionally their **birthday** (month/day, for birthday offers) and **how they heard about you**.
- It creates their customer record (or updates an existing one — matched by email or phone, so no duplicates), shows a friendly **thank-you** screen, and **resets automatically** for the next guest.

- No login is required and the rest of your store stays private.

---

6. Payouts

Pay your consignors. Open **Payouts** from the sidebar.

- **Pending Balances** tab lists consignors who are owed money.
- Select who to pay, choose the **method** (Check, Cash, **Zelle**, **Cash App**, or ACH), and process — checks are numbered sequentially.
- For **Zelle / Cash App / Check**, you'll be asked for the destination (their Zelle phone/email, Cash App $Cashtag, or mailing address). It's saved on the consignor, so next time it's pre-filled. Batch payouts use each consignor's saved details automatically.
- **Sync sold items** (top-right of the Payouts page) credits consignors for any items already marked **SOLD** that aren't yet showing a balance — for example, sales brought in through a data import. Normally selling an item (via the POS or by marking it SOLD) credits the consignor automatically; use this button once after an import, or any time a sold item isn't reflected on a balance.
- **Payout History** tab shows everything you've paid.
- You can **edit a past payout** (amount, method, destination, status, note) from the **Payouts** tab on a consignor's page — changing the amount automatically re-adjusts their balance and ledger.
- Consignors with portal access can also **request a payout** themselves; those requests appear as pending payouts for you to process.

---

7. Contracts

Consignment agreements with e-signature. Open **Contracts** from the sidebar.

1. **New Contract** → pick a consignor → optionally **send the signing link by email**.
2. The consignor opens the link, reads the agreement, **signs on screen**, and submits. Their signature, IP, and timestamp are recorded.
3. The contract's status moves **Draft** → **Sent** → **Signed**.
4. Click the eye icon to view a contract (and its signature).
5. **Resend / fix the email:** click the **paper-plane (Send)** icon on a contract row to re-send the signing link. You'll be asked which email to send to — enter your own to **test**, or **correct the consignor's email** if it was wrong. The consignor opens the link and signs **without any login or password**.
6. **Download / print:** open a contract (eye icon) and click **Download** to get a clean printable copy — the full agreement, the **signature**, and the signer details (name, date, IP). Use your browser's **Save as PDF** or print it.

The agreement text is your store's standard template; the owner can edit it in **Settings** → **Contract Template**.

7a. Admin — employee hours & payouts

Next to **Items** and **Contracts** there's an **Admin** tab (also reachable at **your site** `/admin`). It's a simple log for tracking staff work and pay.

- Each row records a **Date**, **Employee name**, **Phone number**, **Work hours**, and **Payout** (plus an optional note).
- **Add** — click **Add** (top-right) to log a new entry. Fill in the name, date, hours, payout, and optional phone/note, then **Add entry**.
- **Calendar** — a monthly calendar sits above the table.
- **Tap any day** (including future dates) to open the Add form with that date pre-filled, so you can schedule upcoming shifts.
- Each day shows the **employee names** working that day, color-coded. A **color legend** of all employees appears under the calendar.
- **Select many days at once:** click **Select days**, then click the days you want (hold **Shift** and click to pick a whole range). Click **Log hours for N days** and the hours/employee you enter are applied to *every* selected day in one go — great for scheduling a recurring week or a vacation block.
- Use the arrows (or **Today**) to move between months.
- **Edit** — click the pencil on any row to change it, or **Delete** it.
- The cards at the top total the number of **employees**, **hours**, and **payout** across all entries.

---

8. Store Insights & Reports

- **Store Insights** — charts for 30-day revenue, top categories, and inventory status.
- **Reports** — key numbers plus **CSV export** for Sales, Inventory, Consignors, and Ledger (with optional date range). Great for bookkeeping.
- **Email Activity** (on the Reports page) — a store-wide log of every email the platform has sent (inventory, contract, appointment, and Sip & Shop), with the **date & time**, type, recipient, subject, and status (Sent / Test / Failed). (Per-consignor inventory & contract emails also appear on each consignor's page.)

---

9. Online shop

Your public storefront is at **your site** `/shop` — no login needed; this is what customers see. Any item marked **"List on public online shop"** appears there automatically. Share that link on social media or your website.

---

10. Consignor & Dealer portals

- **Consignor portal** (`/portal`) — consignors with portal access sign in to see their items, earnings, and balance, and to **request a payout**.
- **Dealer entry** (`/dealer`) — lets approved dealers submit multiple items at once for your review.

---

11. Importing existing data

Owners/managers can bulk-load data at **your site** `/import` by uploading a prepared file: - A **consignors + inventory** file, or - A **customers** file.

The page shows a preview (counts) before you confirm. Records are matched by email so nothing is duplicated. (Preparing these files from spreadsheets is done with help from your admin.)

Quick tips

- **Scanning works anywhere there's a field** — the scanner just "types" the code and presses Enter.
- **Switch views** (List / Cards / Compact) to suit the task — Compact is great for scanning long lists; Cards are nice for browsing with photos.
- **Multi-select + bulk actions** save a lot of time (markdowns, labels, points).
- Use **Dark mode** for low-light counters.

Questions about setup, accounts, payments, or integrations? See the Admin Guide.

CLASSIC CONSIGNS BY KYS

Admin & Operations Guide

For the owner / administrator

Classic Consigns by KYS — Admin & Operations Guide

For the store owner / administrator. Covers hosting, configuration, accounts, security, integrations, data import/export, backups, and troubleshooting.

---

1. Architecture at a glance

Layer	Technology
App	Next.js 16 (App Router, TypeScript), React 19
Database	PostgreSQL via Prisma 7 (pg driver adapter)
Auth (staff)	NextAuth v5 (JWT sessions, credentials)
Auth (consignor/dealer portal)	separate JWT (`jose`)
Hosting	Railway (app + Postgres + a Volume for photos)
Photo storage	Railway Volume (served via `/api/photos`)
Payments	Clover REST API
Email	Resend
AI item entry	Anthropic Claude (`claude-haiku-4-5`) vision
Barcodes	Code128 labels via JsBarcode

The code lives in the GitHub repo and **auto-deploys to Railway** on every push to the connected branch.

---

2. Hosting on Railway

The project (e.g. "patient-dream") contains two services: 1. **App service** — deployed from the GitHub repo. Has a **Volume** mounted (e.g. at `/data`) for item photos. 2. **Postgres** — the database.

Build/run config lives in `railway.json`: - Build: Nixpacks, `npm run build`. - Start: `npx prisma db push --accept-data-loss && npm run start` (syncs the database schema, then starts the server). - Healthcheck path: `/login`.

> The app binds to Railway's `$PORT` automatically. The Prisma client is created lazily, so the **build** does not require a database connection, but the **running app** and the start command do (via `DATABASE_URL`).

Deploying changes

Push to the connected branch → Railway rebuilds and redeploys automatically. Watch the **Deployments** tab; a healthy deploy ends **Active / green**.

---

3. Environment variables

Set these on the **app service** → **Variables** tab. Required unless noted.

Variable	Purpose
`DATABASE_URL`	Postgres connection. Use the reference `${{Postgres.DATABASE_URL}}`. **Must be set before the first deploy.**
`NEXTAUTH_URL`	The app's public URL, e.g. `https://your-store.up.railway.app`.
`NEXTAUTH_SECRET`	Random secret for staff auth. Generate: `openssl rand -base64 32` (or `node -e "console.log(require('crypto').randomBytes(32).toString('base64'))"`).
`PORTAL_JWT_SECRET`	Random secret for consignor/dealer portal tokens.
`ANTHROPIC_API_KEY`	*Optional.* Enables AI item entry. From console.anthropic.com.
`CLOVER_API_KEY` / `CLOVER_MERCHANT_ID`	*Optional.* Enables Clover card orders.
`CLOVER_API_BASE`	*Optional.* `https://sandbox.dev.clover.com` for sandbox testing; omit for production.
`RESEND_API_KEY` / `EMAIL_FROM`	*Optional.* Enables outbound email (contracts, payouts, appointments, layaway, borrowed).
`BOOKING_NOTIFY_EMAIL`	*Optional.* Where new-appointment alerts go (falls back to store email, then kimberly@classicconsigns.com).
`CRON_SECRET`	*Optional.* Secret that protects `/api/cron/reminders` for the daily layaway/return reminder runner.
`SHOPIFY_WEBHOOK_SECRET`	*Optional.* Verifies Shopify webhooks.
`RESET_TOKEN`	*Optional/temporary.* Enables the `/reset` password-reset page while set. Remove when not in use.

A volume at a mount path (Railway sets `RAILWAY_VOLUME_MOUNT_PATH`) stores photos under `<mount>/uploads`. `UPLOAD_DIR` can override the location.

---

4. First-run setup (new deployment)

1. Set **all required** env vars (above), especially `DATABASE_URL`, `NEXTAUTH_URL`, `NEXTAUTH_SECRET`, `PORTAL_JWT_SECRET`.
2. Attach a **Volume** to the app service (e.g. mount `/data`).
3. Deploy; wait for **Active**.
4. Open **your site** `/api/setup` — it should return `{"needsSetup":true}`.

5. Open **your site** `/setup` → enter store name, your email, a password → **Create my account**. This creates the owner account and some starter categories, then **locks itself**.
6. Sign in at `/login`.

---

5. Accounts & roles

Roles: **OWNER**, **MANAGER**, **EMPLOYEE** (set on the `User` record).

- **Owner/Manager** can change store settings, manage API keys & webhooks, import data, and edit store/Clover settings.
- The first owner is created via `/setup`.
- There is no staff-invite UI yet. To add staff, create a `User` row in the database (see "Direct database access") with the store's `storeId`, a bcrypt-hashed password, and the desired role.

Resetting a password

1. Add a temporary env var `RESET_TOKEN` = any random string.
2. Wait for redeploy, then open **your site** `/reset`.
3. Enter the token, the account email, and a new password → **Update password**.
4. **Remove the** `RESET_TOKEN` **variable** afterward to disable the page.

---

6. Store configuration (Settings)

Settings (sidebar) covers: - **Store Information** — name, email, phone, **tax rate** (entered as %), currency. Used across the app (e.g. POS tax). - **Clover** — merchant ID + API key (stored in the database; env vars also work as a fallback). - **Online Shop** — your public `/shop` URL. - **Categories** — add/manage item categories. - **API Keys** — generate keys for external integrations (shown once). - **Webhooks** — register endpoints to receive events (sale.created, item.sold, contract.signed, etc.), HMAC-signed. - **Discount Rules** — percentage or fixed-amount rules, optional promo codes. - **Contract Template** — edit the consignment agreement. Placeholders: `{{storeName}}`, `{{consignorName}}`, `{{splitPercent}}`.

Admin — employee work log (`/admin`)

The **Admin** tab (next to Items/Contracts, at `/admin`) is a lightweight log of employee **hours and payouts** — date, employee name, phone, work hours, payout, and an optional note. Staff can **Add** entries and **Edit/Delete** existing ones. It's backed by the `WorkLog` table (one row per entry, scoped to the store); no external payroll integration is involved. It is not a payroll processor — it's a record you keep for your own bookkeeping.

Consignment appointments (`/book` public page)

Prospective consignors book drop-off appointments at the **public** page `/book` (no login). Staff manage them under **People** → **Appointments** (`/appointments`).

- Scheduling rules live in `src/lib/appointments.ts` — `SLOT_TIMES` (the four 40-minute Saturday slots), `SLOT_MINUTES` , and `upcomingSaturdays()` . Edit `SLOT_TIMES` there to change times or the number of slots per day.
- Data: `ConsignAppointment` (one row per booked slot; unique per store+date+slot prevents double-booking) and `ConsignBlockedDate` (a blocked Saturday). Both are created by `prisma db push` on deploy.
- `/book` and `/api/book` are in the middleware `PUBLIC_PATHS` allow-list so they're reachable without a login.
- **Booking notifications:** every online booking emails the store. The recipient is `BOOKING_NOTIFY_EMAIL` (env var) if set, otherwise the store email from Settings, otherwise `kimberly@classicconsigns.com` . Set `BOOKING_NOTIFY_EMAIL` to send alerts somewhere specific. The consignor also gets a confirmation if they entered an email.
- **Test email:** People → Appointments has a **Send test email** button (`POST /api/appointments/test-email`) that returns the real Resend result for troubleshooting delivery.

Layaway & Borrowed agreements

Two more People tabs: **Layaway** (`/layaway`) and **Borrowed** (`/borrowed`).

- Data: `Layaway` and `BorrowedGarment` tables (created by `prisma db push`).
- On creation, the customer/borrower is emailed the agreement if an email is provided (layaway → agreement + payment schedule; borrowed → item list + terms).
- **Reminders:** layaway payment reminders fire at **15 / 30 / 45 days** from the start date; borrowed-garment **return reminders** fire on/after the return date. Logic lives in `src/lib/reminders.ts` and is idempotent (each reminder is timestamped once sent).
- Reminders run two ways: 1. **Opportunistically** whenever staff open the Layaway/Borrowed pages (the list endpoints call the processor). Good enough if staff visit regularly. 2. **Daily** via `POST /api/cron/reminders` , protected by the `CRON_SECRET` env var (send `Authorization: Bearer <CRON_SECRET>` or `?secret=`). If `CRON_SECRET` is unset the endpoint is disabled (503).

Sip & Shop events

People tab **Sip & Shop** (`/sip-shop` , staff) plus a **public** request page at `/sip-and-shop` (no login). Model: `SipShopEvent` . Events run 7–10 PM.

- Staff schedule events on a calendar; the host gets a confirmation email (date, 7–10 PM, guests, perks) when an email is provided, and a reminder email ~2 days before (see `SIP_REMINDER_DAYS_BEFORE` in `src/lib/reminders.ts`).
- The public page lets customers request their own date; it creates a `SipShopEvent` , emails the customer a confirmation, and notifies the store (`BOOKING_NOTIFY_EMAIL` → store email → kimberly@classicconsigns.com).

- `/sip-and-shop` and `/api/sip-and-shop` are in the middleware public allow-list.

Kiosk customer sign-in (`/welcome`)

Public kiosk page at `/welcome` (no login) for a tablet to replace the paper guest book. `POST /api/checkin` creates or updates a `Customer` (matched by email or phone to avoid duplicates). Added `Customer.birthday` ("MM-DD") and `Customer.source` fields. `/welcome` and `/api/checkin` are in the middleware public allow-list. Put the tablet in kiosk/Guided-Access mode on this page so customers can't reach the staff dashboard.

To enable the daily runner (recommended): 1. Set a `CRON_SECRET` env var in Railway (any long random string). 2. In GitHub → repo **Settings** → **Secrets and variables** → **Actions**, add two repository secrets: - `APP_URL` = your live site, e.g. `https://data-production-5b03.up.railway.app` - `CRON_SECRET` = the same value as in Railway 3. The workflow `.github/workflows/reminders.yml` runs daily (14:00 UTC) and can be run manually from the **Actions** tab (Run workflow). It no-ops if the secrets aren't set.

---

7. Integrations

Clover (card payments)

Set `CLOVER_API_KEY` + `CLOVER_MERCHANT_ID` (env or Settings). Use `CLOVER_API_BASE=https://sandbox.dev.clover.com` for sandbox testing. **Note:** the app creates Clover **orders**; the actual card capture happens on a Clover **device/terminal** tied to the merchant — full in-app card capture is not yet implemented.

Live Clover transactions on Insights: Store Insights shows a **Clover Card Transactions** panel (today + last 30 days + a recent list) that pulls real payments from Clover and auto-refreshes every 60s. Powered by `GET /api/clover/transactions` → `CloverClient.listPayments` (`/v3/merchants/{mId}/payments`). The API token must have **Payments: Read** permission (Clover dashboard → Setup → API Tokens); otherwise the panel shows a permission error. This is separate from the app's own POS revenue numbers (which only reflect sales rung up inside the app).

Resend (email)

Set `RESEND_API_KEY` and `EMAIL_FROM` (verified domain recommended). Powers contract links, payout notifications, welcome emails, and **appointment confirmations** (sent when a consignor books at `/book` or staff add a booking with an email). If `RESEND_API_KEY` is unset, bookings still work — no email is sent.

Test your email setup with the included script — it sends a sample appointment confirmation so you can confirm the key/domain work before going live:

```
# locally
RESEND_API_KEY=re_xxx EMAIL_FROM="Classic Consigns by KYS <kimberly@classicconsigns.com>" \
  node scripts/send-test-email.mjs you@example.com

# on Railway (RESEND_API_KEY / EMAIL_FROM already set) — from the service shell:
node scripts/send-test-email.mjs you@example.com
```

If Resend returns a domain error, the sender domain isn't verified yet — verify it under Resend → Domains (add the SPF/DKIM DNS records), or point `EMAIL_FROM` at an already-verified address.

DMARC (deliverability): in addition to Resend's SPF/DKIM records, add a DMARC record at the domain's DNS to improve inbox placement and satisfy Gmail/Yahoo sender requirements:

Type	Name	Value
TXT	`_dmarc`	`v=DMARC1; p=none; rua=mailto:kimberly@classicconsigns.com`

`p=none` is monitor-only (no effect on mail flow); it can be tightened to `p=quarantine` later once reports look clean. Verify at whatsmydns.net by looking up `_dmarc.classicconsigns.com` (TXT).

Anthropic / Claude (AI item entry)

Set `ANTHROPIC_API_KEY`. The add-item photo analysis uses `claude-haiku-4-5`. Cost is well under a cent per item. If unset, AI entry simply returns a 503 and everything else works.

Shopify (webhooks, inbound)

Set `SHOPIFY_WEBHOOK_SECRET`; point Shopify order webhooks at `/api/webhooks/shopify`. Incoming orders mark matched items sold and credit the consignor.

Outbound webhooks / API keys

Configure in **Settings**. API requests authenticate with a `Bearer cp_live_…` key; only a SHA-256 hash is stored.

---

8. Importing data

Owners/managers upload prepared JSON at **your site** `/import`: - `{ consignors: [...] }` — creates consignors (matched by email) and their items (Active, priced from the sheet, default 50% split). - `{ customers: [...] }` — creates customers (matched by email).

Both are idempotent (safe to re-run; existing records are skipped). Preparing these files from spreadsheets is typically done by cleaning the sheet into the expected JSON shape; ask the maintainer to generate the import file from a new spreadsheet.

9. Data export & reporting

Reports → **Export** produces CSVs for Sales, Inventory, Consignors, and Ledger (optional date range) for bookkeeping and accounting.

10. Backups & data safety

- **Database:** enable Railway's Postgres **backups/snapshots**. The database holds all consignors, customers, items, sales, ledger, payouts, and contracts.
- **Photos:** the Railway **Volume** holds item photos — enable volume snapshots. Photos are **not** in git or the database.
- **Deletions are protected:** consignors/items with sales history can't be deleted.

> The start command uses `prisma db push --accept-data-loss`. The current schema is additive, so this is a no-op in practice. Before making a **destructive** schema change against live data, take a database backup first, and consider switching to committed Prisma migrations (`prisma migrate`).

11. Direct database access

- Browse/edit data with Prisma Studio locally: `DATABASE_URL="…" npx prisma studio`.
- Railway also provides a **Console** and a database connection string for the Postgres service.

12. Local development

See `LOCAL_DEV.md` for the full local setup (clone, `.env`, `docker compose up` for a local Postgres, `npm run db:push`, `npm run db:seed`, `npm run dev`).

13. Troubleshooting

Symptom	Likely cause / fix
Build fails with `P2038 … driver adapter`	A DB route evaluated at build time without `DATABASE_URL`. The client is lazy now; ensure you're on current code.
"Application failed to respond" after deploy	The start command failed — check **Deploy Logs**. Often the DB migration step or a bad start command.
Every page 500s	Missing `NEXTAUTH_SECRET`, or NextAuth can't trust the host — `trustHost` is enabled; make sure the secret is set.
Login says "invalid email or password" with a correct password	Check the account exists (`/api/setup` → `needsSetup:false`). If needed, reset via `/reset`.
Login succeeds then bounces to `/login`	Session cookie not seen on navigation — the app uses a hard navigation after login to prevent this; ensure current code.
`/api/setup` returns 500	Database has no tables / can't connect. The start command runs `prisma db push`; verify `DATABASE_URL`.
AI item entry returns 503	`ANTHROPIC_API_KEY` not set (optional feature).
Photos don't upload/show	Volume not attached, or `/api/photos` blocked. Ensure a Volume is mounted.
Card payments don't charge	Expected — Clover orders are created, but card capture needs a Clover device.

Reading logs: Railway → app service → latest deployment → **View logs** → **Deploy Logs**. Use the filter box to find specific text.

---

14. Security checklist

- ☐ `NEXTAUTH_SECRET` and `PORTAL_JWT_SECRET` are strong, unique random values.
- ☐ `RESET_TOKEN` is **removed** when not actively resetting a password.
- ☐ Database and Volume backups are enabled.
- ☐ The owner password was changed from any initial value.
- ☐ Secrets are only in Railway Variables (never committed to git).
- ☐ API keys are rotated if exposed.

See `USER_GUIDE.md` for day-to-day staff instructions.